Architecture Evolution History

作成日: 2026年10月 / 対象: 新規参画エンジニア

プロジェクト開発史とアーキテクチャ変遷

～クラシックMVCからクリーンアーキテクチャへの進化の歩み～

本プロジェクトがどのような経緯で始まり、なぜクラシックなMVC（Servlet/JSP/DAO）からクリーンアーキテクチャへと舵を切ったのか。
その歴史的変遷を知ることで、現在のコードが「なぜこう書かれているのか」を深く理解できます。

Overview

変遷の4つのフェーズ（タイムライン）

Phase 1: 立ち上げ期 クラシックMVC & JavaBeans

単純な社員管理と受動的MVCモデル

Servlet + JSP + FrontController + DAO で開発開始。エンティティはgetter/setterのみのJavaBeans（貧血ドメイン）。 社員一覧やログイン等の単純なCRUDは問題なく動作していた。

Phase 2: 課題爆発期 SES特有ルールの複雑化とデグレ

ビジネスルールの肥大化とテスト不能の限界

客先精算幅（140h～180h）、36協定アラート、直接/間接時間、週報多層決裁などの要件が追加。 ロジックがActionやDAOに散らばり、少し修正するたびに既存機能が壊れる（デグレ）。Tomcat起動なしではテスト不能。

Phase 3: 設計刷新期 クリーンアーキテクチャ移行

4層構造への移行と「自己防衛ドメイン」の誕生

社内勉強会を経て、`com.example` を完全撤廃し直下4層（domain, usecase, infrastructure, controller）へ再編。ドメイン層を完全孤立化し、DB・サーバー不要のJUnit単体テスト20件（所要0.8秒）を完備。

Phase 4: 現場適応期 (現在) UI/UX洗練と実務例外の吸収

中抜けスケジュール対応・クリア機能・Tailwind静的化

「休日出勤の常時表示」「誤入力のクリア機能」「客先18時終了・社内19時～20時の中抜け自動計算」など、 現場の生々しい例外運用をドメインの整合性を壊さずに吸収する仕組みを整備。

Code Comparison

コード比較：従来のMVC vs 現在のクリーン構造

✕ 以前のコード（Action/DAO直結・貧血Bean）

デグレ多発・テスト不可

```
// Actionクラスの中にSQLと計算が混在
public class KintaiUpdateAction implements Action {
    public View execute(HttpServletRequest req) {
        // 1. パラメータをそのままBeanにset
        Kintai k = new Kintai();
        k.setStartTime(req.getParameter("start"));
        k.setEndTime(req.getParameter("end"));

        // 2. Actionの中で直接計算
        int hours = calc(k.getStartTime(), k.getEndTime());
        if (hours > 8) { // 業務ルールがActionに露出
            k.setOvertime(hours - 8);
        }

        // 3. DAOに丸投げ
        KintaiDao dao = new KintaiDao();
        dao.update(k); // DBエラー以外は何でも通ってしまう
        return new View("/kintai.jsp");
    }
}
```

○ 現在のコード（4層クリーンアーキテクチャ）

自己防衛・秒速テスト可能

```
// Actionは薄く、UsecaseとDomainが自己防衛
public class SesAttendanceApiAction implements Action {
    public View execute(HttpServletRequest req) {
        // 1. UsecaseのInputに変換して委譲するだけ
        FillAttendanceUseCase useCase =
            new FillAttendanceUseCase(attendanceRepo);

        // 2. ドメインエンティティ内部で不変条件を厳格に自己検証
        //  （拘束時間、中抜け休憩、直接/間接の整合性、ロック状態）
        DailyAttendance att = useCase.execute(input);

        // 3. 結果をJSONで返却
        return sendJson(att);
    }
}
```

新しくプロジェクトに入る方へ

本プロジェクトのアーキテクチャは、「流行りだから」導入されたものではありません。
「複雑化するSESの労務・契約ルールを、バグを出さずに安全に拡張し続けるための必然」として進化してきました。 新しい機能を追加する際も、この歴史的背景を踏まえ、「ドメインの純粋性と自己防衛」を維持してください。